```
sim = require('sim')

-- ===== settings =====
WHEEL_RADIUS = 0.30     -- 0.6 m wheels
TRACK        = 0.60     -- distance between left and right wheel centres
(m)
WHEEL_TORQUE = 100      -- N?m per wheel motor (6 wheels)
FRONT_EDGE   = 0.95     -- front wheel tips, from robot centre (m)
REAR_EDGE    = 0.95     -- rear wheel tips, from robot centre (m)

-- speeds
DRIVE_SPEED  = 0.60     -- driving to a container (m/s)
CARRY_SPEED  = 0.50     -- driving while carrying (m/s)
BACK_SPEED   = 0.35     -- all reversing (m/s)
DETOUR_SPEED = 0.40     -- driving past an obstacle (m/s)
SLOW_SPEED   = 0.20     -- speed when a wall/obstacle is close ahead
(m/s)
TURN_SPEED   = 1.0      -- turning on the spot (rad/s)
SLOW_TURN    = 0.6      -- turning in tight spaces (rad/s)

-- obstacles
TURN_SWEEP   = 1.15     -- known obstacles closer than this block a spin
on the spot (m)
CORRIDOR     = 0.75     -- obstacles closer than this to the driving line
get bypassed (m)
BYPASS_GAP   = 1.05     -- pass bypassed obstacles at this sideways
distance (m)
AVOID_DIST   = 0.4      -- straight-ahead rays: react closer than this
(m)
SIDE_DIST    = 0.3      -- front-corner rays
DIAG_DIST    = 0.1      -- diagonal rays
DOOR_DIAG_DIST = 0.1    -- diagonal rays while getting the room container
(narrow doorway)

-- ===== timing with the walking man =====
-- Step 1: MEASURE_RUN = true  -> drives exactly like the old 0.25 code
and prints
--         'LEAVING for Biohazard10 at t=... s'. Write that number into
LEAVE_AT.
-- Step 2: MEASURE_RUN = false -> uses 0.1 everywhere; after the drop
before Biohazard10 it stands
--         still until LEAVE_AT, so it sets off at the same moment as the
0.25 code did.
MEASURE_RUN   = true
LEAVE_AT      = nil                  -- e.g. 675.55 (s, from step 1)
MAN_CONTAINER = 'Biohazard10'        -- the container inside the walking
man's loop
MAN_NAME      = 'Bill[0]'            -- the walking man: for Biohazard10
don't drive round him (that goes past it), wait instead
MAN_BACKUP    = 10.0                 -- after picking Biohazard10:
reverse straight this far (m), then carry on as normal
MAN_CYCLE     = 2*(31.5/0.56 + 3)    -- his walk repeats every ~118.5 s
(31.5 m path, 0.56 m/s, 3 s stop at each end)
if MEASURE_RUN then DIAG_DIST = 0.25; LEAVE_AT = nil end
SLOWDOWN     = 2.0      -- start slowing at AVOID_DIST x this (m)
REAR_STOP    = 0.4      -- don't reverse if something is closer behind
(m)
```

```
TURN_CLEAR   = 0.2      -- rays: need this much space front and back
before spinning (m)

-- distances
AVOID_BACKUP = 3.0      -- on an obstacle: reverse this far first (m)
AVOID_TURN   = 1.2      -- ...then turn this much (rad, about 70 deg)
DETOUR_DIST  = 1.5      -- ...then drive forward this far (m)
MAKEROOM     = 2.0      -- reverse this far to make room before turning
(m)
REPOS_DIST   = 2.2      -- misaligned with a container: move to this
distance from it (m)
PICK_BACKUP  = 3.0      -- after lifting a container: reverse straight
this far before turning (m)
CARRYBACK_DIST = 2.5    -- off-centre at the drop spot: move to this
distance from it (m)
FINAL_APPROACH = 1.5    -- within this distance of the drop spot,
approach slowly (m)
SLOT_SPACING = 1.2      -- distance between containers in the green area
(m)
SLOT_MARGIN  = 0.6      -- keep containers this far from the green area
edge (m)
MIN_BACKUP   = 5.0      -- after dropping, reverse straight at least this
far (m)
EXIT_CLEAR   = 1.5      -- ...and until this far outside the green area
(m)
EXIT_STOP    = 0.3      -- while backing straight out, stop only if
something is this close behind (m)
STUCK_TIME   = 5.0      -- seconds without moving forward before treating
it as an obstacle
-- walking people (Bill[0] walks his path at 0.56 m/s). People who never
move are just obstacles.
WALKER_NEAR  = 2.5      -- a walking person this close: don't reverse
towards them / don't swing round (m)
WALKER_SAFE  = 4.0      -- before going in to grab: the walker must be at
least this far from the container (m)
WALKER_WATCH = 9.0      -- ...and if he is closer than this and still
coming towards it, wait until he has passed (m)
WALKER_MAXWAIT = 40     -- never wait longer than this in one go (s)

-- the map (from OuterWall1..4): x -25.75 .. 24.25, y -26.75 .. 23.25
ARENA = {-25.75, 24.25, -26.75, 23.25}
STAGE_DIST   = 3.0      -- green area: line up this far out from the drop
spot, on the open side, then drive straight in (m)

-- steps
CLIMB_PITCH  = 0.13     -- tilt (sin of angle, about 7.5 deg) that means
"on a step"
CLIMB_UP     = 0.30     -- speed going up a step (m/s)
CLIMB_DOWN   = 0.15     -- speed going down a step (m/s)
CLIMB_MAX    = 10       -- give up climb mode after this many seconds

TERRAIN = {'step', 'floor', 'ramp', 'slope', 'terrain', 'safezone',
'plane', 'water'}
PEOPLE  = {'bill', 'mannequin', 'human', 'person', 'casualty'}

SENSOR_NAMES = {
    F   = 'Front_proximitySensor',
    FL  = 'Left_proximitySensor',
```

```
    FR  = 'Right_proximitySensor',
    LF  = 'Low_front_proximitySensor',
    LL  = 'low_Front_left_proximitySensor',
    LR  = 'low_Front_right_proximitySensor',
    DFL = 'Diag_Front_left_proximitySensor',
    DFR = 'Diag_Front_Right_proximitySensor',
    SL  = 'Side_left_proximitySensor',
    SR  = 'Right_side_proximitySensor',
    BL  = 'Diag_Rear_left_proximitySensor',
    BR  = 'Diag_Rear_right_proximitySensor',
    B   = 'Rear_proximitySensor',
    LB  = 'Low_rear_middle_proximitySensor',
    LBL = 'Low_rear_left_proximitySensor',
    LBR = 'Low_rear_right_proximitySensor',
    LSL = 'Low_left_side_proximitySensor',
    LSR = 'Low_right_side_proximitySensor',
}
LOW = {LF=true, LL=true, LR=true, LB=true, LBL=true, LBR=true, LSL=true,
LSR=true}

FRONT_CHECKS = {{'F', AVOID_DIST}, {'LF', AVOID_DIST},
                {'FL', SIDE_DIST}, {'FR', SIDE_DIST}, {'LL', SIDE_DIST},
{'LR', SIDE_DIST},
                {'DFL', DIAG_DIST}, {'DFR', DIAG_DIST}}
FRONT_KEYS = {'F', 'FL', 'FR', 'LF', 'LL', 'LR', 'DFL', 'DFR'}
REAR_KEYS  = {'B', 'BL', 'BR', 'LB', 'LBL', 'LBR'}
LEFT_KEYS  = {'SL', 'LSL', 'FL', 'LL', 'DFL'}
RIGHT_KEYS = {'SR', 'LSR', 'FR', 'LR', 'DFR'}
CASUALTY_PAIRS = {
    {{'LF', 'LL', 'LR'}, {'F', 'FL', 'FR'}},
    {{'LSL'}, {'SL'}},
    {{'LSR'}, {'SR'}},
    {{'LB', 'LBL', 'LBR'}, {'B', 'BL', 'BR'}},
}
CLIMB_STATES = {['goto']=true, ['carry']=true, ['bypass']=true,
['travel']=true}

-- NEW: the two extra containers, collected after the other 8
BUILDING_CONTAINER = 'Biohazard7'   -- beside the tall building
ROOM_CONTAINER     = 'Biohazard4'   -- inside the room with the sliding
door (collected last)
APPROACH_DIST      = 3.0            -- stop this far out from the
building side, in line with the container (m)
-- from your scene: Biohazard7 (-20, 10) is on the left side of the
building (building at -13.9, 13.2),
-- 1.4 m from its wall. The robot lines up 3.5 m in front of it, on the
open side, and drives straight in
-- along the building wall.
BUILDING_APPROACH  = {-20.0, 6.5}
DOOR_NAME          = 'slidingDoor'  -- the door model in your scene
DOOR_CENTRE        = {5.75, 10.0}   -- slidingDoor position (x, y)
DOOR_OUT           = 2.5            -- wait this far in front of the door
(m)
-- Biohazard4 (6.0, 12.0) is just inside the door: once the door opens
the robot drives straight in
-- and picks it up with the normal code, then backs straight out through
the door.
```

```
-- time allowed to move a given distance at BACK_SPEED (+5 s spare)
function backTime(d) return d / BACK_SPEED + 5 end

-- ===== helpers =====
function limit(x, a, b) return math.max(a, math.min(b, x)) end
function sgn(x) return (x >= 0) and 1 or -1 end
function wrapAngle(a)
    while a > math.pi do a = a - 2*math.pi end
    while a < -math.pi do a = a + 2*math.pi end
    return a
end
function errText(e) return tostring(e) end
function norm(s) return (string.gsub(string.lower(s), '[^%w]', '')) end
function dist2D(a, b) return math.sqrt((a[1]-b[1])^2 + (a[2]-b[2])^2) end

function tryObj(name)
    local ok, h = pcall(sim.getObject, '/' .. name)
    if ok and h and h ~= -1 then return h end
    return nil
end
function needObj(name)
    local h = tryObj(name)
    if not h then error('OBJECT NOT FOUND: ' .. name) end
    return h
end
function inTree(h, root)
    local n = 0
    while h and h ~= -1 and n < 50 do
        if h == root then return true end
        h = sim.getObjectParent(h); n = n + 1
    end
    return false
end
function rootOf(h)
    local n = 0
    while n < 50 do
        local p = sim.getObjectParent(h)
        if p == -1 then return h end
        h = p; n = n + 1
    end
    return h
end
function aliasOf(h)
    local ok, a = pcall(sim.getObjectAlias, h)
    return ok and a or tostring(h)
end
function nameHas(h, words)
    for _, x in ipairs({h, rootOf(h)}) do
        local a = string.lower(aliasOf(x))
        for _, w in ipairs(words) do
            if string.find(a, w, 1, true) then return true end
        end
    end
    return false
end
function isTerrain(h) return nameHas(h, TERRAIN) end
function isContainer(h) return nameHas(h, {'biohazard'}) end
```

```
DET = sim.objectspecialproperty_detectable_all or
sim.objectspecialproperty_detectable
function setDetectable(h, on)
    pcall(function()
        local p = sim.getObjectSpecialProperty(h)
        if on then p = p | DET else p = p & (~DET) end
        sim.setObjectSpecialProperty(h, p)
    end)
end
function setTreeDetectable(root, on)
    for _, h in ipairs(sim.getObjectsInTree(root, sim.handle_all, 0)) do
setDetectable(h, on) end
end

function inZone(x, y, extra)
    extra = extra or 0
    return math.abs(x - zp[1]) < ZONE_HALF + extra and math.abs(y -
zp[2]) < ZONE_HALF + extra
end
function robotPos() return sim.getObjectPosition(body, WORLD) end
function robotYaw() return sim.getObjectOrientation(body, WORLD)[3] end
function toWorld(x, y)                      -- robot-frame point -> world
point
    local m = sim.getObjectMatrix(body, WORLD)
    local w = sim.multiplyVector(m, {x, y, 0})
    return {w[1], w[2]}
end
function toRobot(x, y)                      -- world point -> robot frame
(ahead, left)
    local p, yaw = robotPos(), robotYaw()
    local dx, dy = x - p[1], y - p[2]
    return math.cos(yaw)*dx + math.sin(yaw)*dy, -math.sin(yaw)*dx +
math.cos(yaw)*dy
end

-- ===== attach / detach the container to the palm =====
function attachContainer(h)
    contShapes = sim.getObjectsInTree(h, sim.object_shape_type, 0)
    saved = {}
    for _, s in ipairs(contShapes) do
        saved[s] = {sim.getObjectInt32Param(s, sim.shapeintparam_static),
                    sim.getObjectInt32Param(s,
sim.shapeintparam_respondable)}
        sim.setObjectInt32Param(s, sim.shapeintparam_static, 1)
        sim.setObjectInt32Param(s, sim.shapeintparam_respondable, 0)
        sim.resetDynamicObject(s)
    end
    sim.setObjectParent(h, palm, true)
    setTreeDetectable(h, false)
    attached = true
end
function detachContainer(h)
    if not attached then return end
    sim.setObjectParent(h, -1, true)
    for _, s in ipairs(contShapes) do
        sim.setObjectInt32Param(s, sim.shapeintparam_static, saved[s][1])
        sim.setObjectInt32Param(s, sim.shapeintparam_respondable,
saved[s][2])
```

```
        sim.resetDynamicObject(s)
    end
    setTreeDetectable(h, true)
    attached = false
end

-- ===== NEW: the two extra containers (collected after the other 8)
=====
-- Biohazard7: drive to the open side of the building, then the normal
pick-up code collects it.
-- Biohazard4: drive to the door, wait for it to open, go in, normal
pick-up, come back out, normal carry.
function worldBox(h)                          -- world-aligned box around a
shape {x1, x2, y1, y2, z1, z2}
    local ok, r = pcall(function()
        local P = {}
        for i = 0, 5 do P[i] = sim.getObjectFloatParam(h,
sim.objfloatparam_objbbox_min_x + i) end
        local m = sim.getObjectMatrix(h, sim.handle_world)
        local b = {1e9, -1e9, 1e9, -1e9, 1e9, -1e9}
        for _, x in ipairs({P[0], P[3]}) do for _, y in ipairs({P[1],
P[4]}) do for _, z in ipairs({P[2], P[5]}) do
            local w = sim.multiplyVector(m, {x, y, z})
            b[1] = math.min(b[1], w[1]); b[2] = math.max(b[2], w[1])
            b[3] = math.min(b[3], w[2]); b[4] = math.max(b[4], w[2])
            b[5] = math.min(b[5], w[3]); b[6] = math.max(b[6], w[3])
        end end end
        return b
    end)
    if ok then return r end
    return nil
end
function boxDist(q, b)
    local dx = math.max(b[1] - q[1], 0, q[1] - b[2])
    local dy = math.max(b[3] - q[2], 0, q[2] - b[4])
    return math.sqrt(dx*dx + dy*dy)
end
function boxesTouch(a, b, g)
    return a[1] - g < b[2] and b[1] - g < a[2] and a[3] - g < b[4] and
b[3] - g < a[4]
end
function tallShapes()                         -- walls / buildings (not the
robot, containers, people, floor)
    local out = {}
    for _, h in ipairs(sim.getObjectsInTree(sim.handle_scene,
sim.object_shape_type, 0)) do
        if not inTree(h, body) and not isContainer(h) and not nameHas(h,
PEOPLE) and not isTerrain(h) then
            local b = worldBox(h)
            if b and b[6] > 0.25 then table.insert(out, b) end
        end
    end
    return out
end
function isBig(b) return (b[2] - b[1]) > 15 or (b[4] - b[3]) > 15 end   -
- outer walls / floor
function growBlock(seed, shapes)              -- join everything touching
the seed into one block
```

```
    local blk = {seed[1], seed[2], seed[3], seed[4]}
    local used, changed = {}, true
    while changed do
        changed = false
        for i, b in ipairs(shapes) do
            if not used[i] and not isBig(b) and b[6] > 1.0 and
boxesTouch(blk, b, 0.3) then
                used[i], changed = true, true
                blk = {math.min(blk[1], b[1]), math.max(blk[2], b[2]),
math.min(blk[3], b[3]), math.max(blk[4], b[4])}
            end
        end
    end
    return blk
end
function segHitsBox(a, b, box, g)           -- does the straight line a-
>b pass through the box (grown by g)?
    local t0, t1 = 0, 1
    for k = 1, 2 do
        local o, d = a[k], b[k] - a[k]
        local lo, hi = box[2*k - 1] - g, box[2*k] + g
        if math.abs(d) < 1e-9 then
            if o < lo or o > hi then return false end
        else
            local u, v = (lo - o)/d, (hi - o)/d
            if u > v then u, v = v, u end
            t0, t1 = math.max(t0, u), math.min(t1, v)
            if t0 > t1 then return false end
        end
    end
    return true
end
function nextWaypoint(p, goal)              -- straight to the goal, or
to a corner of the first block in the way
    local hitB, hitT = nil, 1e9
    local all = {}
    for _, b in ipairs(blocks or {}) do table.insert(all, b) end
    for _, b in ipairs(walkerAreas()) do table.insert(all, b) end
    for _, b in ipairs(all) do
        if boxDist(p, b) > 0.3 and boxDist(goal, b) > 0.3 and
segHitsBox(p, goal, b, 0.9) then
            local t = dist2D(p, {(b[1] + b[2])/2, (b[3] + b[4])/2})
            if t < hitT then hitB, hitT = b, t end
        end
    end
    if not hitB then return goal end
    local b, g = hitB, 1.8
    local C = {{b[1] - g, b[3] - g}, {b[2] + g, b[3] - g}, {b[2] + g,
b[4] + g}, {b[1] - g, b[4] + g}}
    local function okPt(c)
        if floorBox and not (c[1] > floorBox[1] + 1 and c[1] <
floorBox[2] - 1 and c[2] > floorBox[3] + 1 and c[2] < floorBox[4] - 1)
then return false end
        for _, o in ipairs(all) do if boxDist(c, o) < 0.9 then return
false end end
        return true
    end
    local best, bestL = nil, 1e9
```

```
    for i = 1, 4 do                                  -- one corner gets us
past
        local c = C[i]
        if okPt(c) and dist2D(p, c) > 0.5 and not segHitsBox(p, c, b,
0.9) and not segHitsBox(c, goal, b, 0.9) then
            local L = dist2D(p, c) + dist2D(c, goal)
            if L < bestL then best, bestL = c, L end
        end
    end
    if best then return best end
    for i = 1, 4 do                                  -- two corners: go to
the first one now
        for _, j in ipairs({i % 4 + 1, (i + 2) % 4 + 1}) do
            local a, c = C[i], C[j]
            if okPt(a) and okPt(c) and dist2D(p, a) > 0.5 and not
segHitsBox(p, a, b, 0.9) and not segHitsBox(c, goal, b, 0.9) then
                local L = dist2D(p, a) + dist2D(a, c) + dist2D(c, goal)
                if L < bestL then best, bestL = a, L end
            end
        end
    end
    return best or goal
end
function startTravel(goal, blk, ret)
    tr = {goal = goal, blk = blk, ret = ret, entered = -1}
    setState('travel')
end
function doorOpen()                              -- open, and stayed open for
2 s (so the panels are out of the way)
    local t = sim.getSimulationTime()
    local open = (door.joint and
math.abs(sim.getJointPosition(door.joint) - door.j0) > 0.2) or (not
door.joint and t - tState > 5)
    if not open then door.openSince = nil; return false end
    door.openSince = door.openSince or t
    return t - door.openSince > 2.0
end
function carryHome()                             -- go round buildings to the
open side of the drop spot, then the normal carry drives straight in
    startTravel({dropSlot.x + ENTRY[1]*STAGE_DIST, dropSlot.y +
ENTRY[2]*STAGE_DIST}, nil, 'carry')
end
function afterRetreat()                          -- normal carry for the 8;
extra ones go round buildings / out the door first
    if insideRoom then
        doorDir, doorRet = 'out', 'carryhome'
        startTravel(door.inn, nil, 'doorface')
    else
        carryHome()
    end
end

function sideIsClear(q, a, shapes, self)   -- 3 m straight out from the
container in direction a: nothing in the way?
    for r = 1.2, APPROACH_DIST + 0.6, 0.2 do
        local x, y = q[1] + math.cos(a)*r, q[2] + math.sin(a)*r
        for _, b in ipairs(shapes) do
            if boxDist({x, y}, b) < 0.55 then return false end
```

```
        end
        if floorBox and not (x > floorBox[1] + 0.8 and x < floorBox[2] -
0.8 and y > floorBox[3] + 0.8 and y < floorBox[4] - 0.8) then return
false end
        for _, o in ipairs(containers) do
            if o ~= self and dist2D(sim.getObjectPosition(o.h,
sim.handle_world), {x, y}) < 0.9 then return false end
        end
    end
    return true
end
function openRoom(q, a, shapes)            -- how far you can drive out
in direction a with a wide berth (m)
    for r = 1.2, 8, 0.2 do
        local x, y = q[1] + math.cos(a)*r, q[2] + math.sin(a)*r
        for _, b in ipairs(shapes) do if boxDist({x, y}, b) < 1.0 then
return r end end
    end
    return 8
end
function setupExtras()
    local shapes = tallShapes()
    floorBox = ARENA                                    -- the outer walls
    for _, c in ipairs(containers) do
        if c.name == BUILDING_CONTAINER then
            local q = sim.getObjectPosition(c.h, sim.handle_world)
            local near, nd = nil, 3.0
            for _, b in ipairs(shapes) do
              if not isBig(b) and b[6] > 1.0 then
                local d = boxDist(q, b)
                if d < nd then near, nd = b, d end
              end
            end
            if near then
                c.block = growBlock(near, shapes)
                local b = c.block
                local dx = (q[1] < b[1]) and (q[1] - b[1]) or ((q[1] >
b[2]) and (q[1] - b[2]) or 0)
                local dy = (q[2] < b[3]) and (q[2] - b[3]) or ((q[2] >
b[4]) and (q[2] - b[4]) or 0)
                if dx == 0 and dy == 0 then dx, dy = q[1] - (b[1] +
b[2])/2, q[2] - (b[3] + b[4])/2 end
                -- straight out from the building side; if that is
blocked, the nearest clear direction
                local a0, best, bestCost = math.atan2(dy, dx), nil, 1e9
                for k = 0, 23 do
                    local a = a0 + k * math.pi / 12
                    local cost = math.abs(wrapAngle(a - a0)) + (8 -
openRoom(q, a, shapes)) * 0.02   -- prefer open space
                    if cost < bestCost and sideIsClear(q, a, shapes, c)
then best, bestCost = a, cost end
                end
                best = best or a0
                c.approach = BUILDING_APPROACH or {q[1] +
math.cos(best)*APPROACH_DIST, q[2] + math.sin(best)*APPROACH_DIST}
                print(string.format('%s is beside a building: will drive
to x=%.2f y=%.2f first, then pick it up',
                    c.name, c.approach[1], c.approach[2]))
```

```
            else
                c.approach = BUILDING_APPROACH
            end
        end
    end

    blocks = {}                                         -- buildings / rooms
as simple boxes, for planning a route
    local used = {}
    for i, b in ipairs(shapes) do
        if not isBig(b) and b[6] > 1.0 then
            local inside = false
            for _, o in ipairs(blocks) do if boxesTouch(o, b, 0) then
inside = true end end
            if not inside then
                local blk = growBlock(b, shapes)
                local p0 = robotPos()
                if boxDist(p0, blk) > 0 and not (zp[1] > blk[1] and zp[1]
< blk[2] and zp[2] > blk[3] and zp[2] < blk[4]) then
                    table.insert(blocks, blk)
                end
            end
        end
    end
    print(#blocks .. ' buildings/rooms found for route planning')

    local rc
    for _, c in ipairs(containers) do if c.name == ROOM_CONTAINER then rc
= c end end
    local dm = tryObj(DOOR_NAME)
    if not (dm and rc) then print('door (' .. DOOR_NAME .. ') or ' ..
ROOM_CONTAINER .. ' not found: no door route'); return end
    local q = sim.getObjectPosition(rc.h, sim.handle_world)
    local dp = sim.getObjectPosition(dm, sim.handle_world)
    local cx, cy = dp[1], dp[2]
    if DOOR_CENTRE then cx, cy = DOOR_CENTRE[1], DOOR_CENTRE[2] end
    -- the way through the door is along the door's thin side
    local m = sim.getObjectMatrix(dm, sim.handle_world)
    local nx, ny = m[1], m[5]                                   -- door's
own x axis in the world
    local okb, sx = pcall(sim.getObjectFloatParam, dm,
sim.objfloatparam_objbbox_max_x)
    local okc, sy = pcall(sim.getObjectFloatParam, dm,
sim.objfloatparam_objbbox_max_y)
    if okb and okc and sx > sy then nx, ny = m[2], m[6] end     -- thin
side is its y axis instead
    local L = math.sqrt(nx*nx + ny*ny); nx, ny = nx/L, ny/L
    if (q[1] - cx)*nx + (q[2] - cy)*ny > 0 then nx, ny = -nx, -ny end
-- n points out of the room
    -- the door's joint (to see when it is open) and its sensors (for
their height)
    local j, dsens = nil, {}
    for _, h in ipairs(sim.getObjectsInTree(dm, sim.handle_all, 0)) do
        local okt, typ = pcall(sim.getObjectType, h)
        if okt and typ == sim.object_joint_type and not j then j = h end
        if okt and typ == sim.object_proximitysensor_type then
table.insert(dsens, h) end
    end
```

```
    door = {c = {cx, cy}, nx = nx, ny = ny, joint = j, j0 = j and
sim.getJointPosition(j) or 0,
            out = {cx + nx*DOOR_OUT, cy + ny*DOOR_OUT}, sens = dsens}
    door.block = nil
    for _, b in ipairs(blocks) do if boxDist({cx, cy}, b) < 0.5 then
door.block = b end end
    rc.inRoom = true
    print(string.format('door at x=%.2f y=%.2f, robot waits at x=%.2f
y=%.2f, joint %s, %d door sensor(s): %s is collected last',
        cx, cy, door.out[1], door.out[2], j and 'found' or 'NOT found',
#dsens, rc.name))

    -- the door sensors lie on the floor and only look 0.10 m up (they
see feet):
    -- so the see-through box sits at the bottom, from 2 cm to 32 cm
above the floor under the robot
    for _, s in ipairs(door.sens) do
        print(string.format('door sensor at height %.2f m',
sim.getObjectPosition(s, sim.handle_world)[3]))
    end
    local ground = sim.getObjectPosition(leftWheels[1],
sim.handle_world)[3] - WHEEL_RADIUS   -- floor under the robot
    local base, h = ground + 0.02, 0.30
    local ok, box = pcall(sim.createPrimitiveShape,
sim.primitiveshape_cuboid, {FRONT_EDGE + REAR_EDGE, 0.9, h}, 0)
    if ok and box then
        trigger = box
        pcall(sim.setObjectAlias, box, 'DoorTrigger')
        pcall(sim.setShapeColor, box, nil,
sim.colorcomponent_ambient_diffuse, {1, 0.5, 0})
        pcall(sim.setShapeColor, box, nil,
sim.colorcomponent_transparency, {0.7})
        sim.setObjectInt32Param(box, sim.shapeintparam_static, 1)
        sim.setObjectInt32Param(box, sim.shapeintparam_respondable, 0)
        local p = robotPos()
        if not pcall(sim.setObjectPosition, box, {p[1], p[2], base +
h/2}, sim.handle_world) then
            pcall(sim.setObjectPosition, box, sim.handle_world, {p[1],
p[2], base + h/2})
        end
        if not pcall(sim.setObjectOrientation, box, {0, 0, robotYaw()},
sim.handle_world) then
            pcall(sim.setObjectOrientation, box, sim.handle_world, {0, 0,
robotYaw()})
        end
        sim.setObjectParent(box, body, true)
        setDetectable(box, true)                    -- detectable all
the time
    end
end
function sysCall_cleanup()
    if trigger then pcall(sim.removeObjects, {trigger}) end
end

-- ===== init =====
function initAll()
    body = needObj('Robot_body')
    palm = needObj('Palm')
```

```
    lift = needObj('left_joint')
    fR   = needObj('Right_finger_joint')
    fL   = needObj('Left_finger_joint')
    leftWheels  = {needObj('Left_forward_joint'),
needObj('Left_backward_joint')}
    rightWheels = {needObj('Right_forward_joint'),
needObj('Right_backward_joint')}
    local lm, rm = tryObj('Left_middle_joint'),
tryObj('Right_middle_joint')
    if lm then table.insert(leftWheels, lm) else print('WARNING:
Left_middle_joint not found') end
    if rm then table.insert(rightWheels, rm) else print('WARNING:
Right_middle_joint not found') end
    print((#leftWheels + #rightWheels) .. ' wheels driven')

    local index = {}
    for _, h in ipairs(sim.getObjectsInTree(body, sim.handle_all, 0)) do
        index[norm(aliasOf(h))] = h
    end
    sens = {}
    local count = 0
    for k, name in pairs(SENSOR_NAMES) do
        local h = index[norm(name)]
        if h then sens[k] = h; count = count + 1
        else print('sensor missing (skipped): ' .. name) end
    end
    print(count .. ' sensors found')
    if not sens.F then error('OBJECT NOT FOUND: Front_proximitySensor')
end

    zone = tryObj('SafeZone') or tryObj('Plane') or tryObj('Safe_zone')
or tryObj('GreenZone')
    if not zone then error('GREEN AREA NOT FOUND: rename it to SafeZone')
end

    for _, j in ipairs(leftWheels) do
        if not pcall(sim.setJointTargetForce, j, WHEEL_TORQUE) then
pcall(sim.setJointMaxForce, j, WHEEL_TORQUE) end
    end
    for _, j in ipairs(rightWheels) do
        if not pcall(sim.setJointTargetForce, j, WHEEL_TORQUE) then
pcall(sim.setJointMaxForce, j, WHEEL_TORQUE) end
    end
    print('wheel torque ' .. WHEEL_TORQUE .. ' N.m, wheel radius ' ..
WHEEL_RADIUS .. ' m')

    setTreeDetectable(body, false)

    WORLD = sim.handle_world
    R, T = WHEEL_RADIUS, TRACK
    HOLD_X = 0.60
    CLOSE_POS = 0.08

    local okI, cyc, iv = pcall(sim.getJointInterval, lift)
    if okI and type(iv) == 'table' then LIFT_POS = iv[1] + iv[2] - 0.005
else LIFT_POS = 0.28 end
    print(string.format('lift max height %.3f m', LIFT_POS))
```

```
    zp = sim.getObjectPosition(zone, WORLD)
    local zw = 3.0
    local ok, bb = pcall(sim.getShapeBB, zone)
    if ok and type(bb) == 'table' and bb[1] and bb[2] then zw =
math.min(bb[1], bb[2]) end
    ZONE_HALF = zw / 2
    print(string.format('Green area at x=%.2f y=%.2f, size %.2f m',
zp[1], zp[2], zw))

    slots = {}
    local m = math.max(ZONE_HALF - SLOT_MARGIN, 0)
    local n = math.max(1, math.floor(2*m/SLOT_SPACING) + 1)
    for i = 0, n-1 do
        for j = 0, n-1 do
            local ox = (n == 1) and 0 or (-m + i*2*m/(n-1))
            local oy = (n == 1) and 0 or (-m + j*2*m/(n-1))
            table.insert(slots, {x=zp[1] + ox, y=zp[2] + oy, used=false})
        end
    end
    print(#slots .. ' drop-off spots in the green area')
    do
        local cx, cy = (ARENA[1] + ARENA[2])/2 - zp[1], (ARENA[3] +
ARENA[4])/2 - zp[2]
        local L = math.max(math.sqrt(cx*cx + cy*cy), 0.01)
        ENTRY = {cx/L, cy/L}                            -- the open side
of the green area
    end

    containers = {}
    for i = 0, 20 do
        local name = (i == 0) and 'Biohazard' or ('Biohazard' .. i)
        local h = tryObj(name)
        if h then
            setTreeDetectable(h, true)
            local p = sim.getObjectPosition(h, WORLD)
            local already = inZone(p[1], p[2])
            table.insert(containers, {h=h, name=name, done=already,
                                      extra=(name == BUILDING_CONTAINER
or name == ROOM_CONTAINER)})
            print(string.format('found %s at x=%.2f y=%.2f%s',
                name, p[1], p[2], already and ' (already in green area)'
or ''))
        end
    end
    if #containers == 0 then error('no containers named Biohazard /
Biohazard1..20') end

    people = {}
    local byRoot = {}
    for _, h in ipairs(sim.getObjectsInTree(sim.handle_scene,
sim.object_shape_type, 0)) do
        if not inTree(h, body) and nameHas(h, PEOPLE) then
            local r = rootOf(h)
            if not byRoot[r] then
                byRoot[r] = {name=aliasOf(r), hs={}, root=r}
                table.insert(people, byRoot[r])
                setTreeDetectable(r, true)
            end
```

```
            table.insert(byRoot[r].hs, h)
        end
    end
    for _, p in ipairs(people) do print('person found: ' .. p.name .. '
(' .. #p.hs .. ' parts)') end

    door, trigger, tr, insideRoom = nil, nil, nil, false
    waitSince = nil
    setupExtras()
    local b7 = tryObj(BUILDING_CONTAINER)
    if b7 then setTreeDetectable(b7, true) end      -- Biohazard7 is
detectable by the sensors, like the others

    records, casualties = {}, {}
    cur = nil
    attached = false
    S, OB = {}, {}
    state, tState, tTarget, lastP, lastC = 'start', 0, 0, 0, 0
    avoidReturn, avoidDir, dirLock = 'goto', 1, 0
    av, mr, bp = nil, nil, nil
    graceUntil = 0
    movedFwd = false
    stuckT = 0
    stuckP = robotPos()
    dropPos, pickPos = nil, nil
    climbing, climbT0 = false, 0
end

function sysCall_init()
    ready = false
    local ok, err = xpcall(initAll, errText)
    if ok then ready = true; print('init OK')
    else print('INIT ERROR: ' .. tostring(err)) end
end

-- ===== sensing =====
function readSensor(k)
    local s = sens[k]
    if not s then return nil end
    local r, d, pt, obj = sim.readProximitySensor(s)
    if not (r and r > 0 and obj and obj ~= -1) then return nil end
    if inTree(obj, body) then return nil end
    if cur and inTree(obj, cur.h) and (state ~= 'travel' or attached)
then return nil end   -- while travelling it IS an obstacle
    if LOW[k] and isTerrain(obj) then return nil end
    return {d=d, obj=obj, pt=pt, s=s}
end
function readAll()
    local o = {}
    for k, _ in pairs(sens) do o[k] = readSensor(k) end
    return o
end
function D(k) return S[k] and S[k].d or nil end
function minD(keys)
    local m = 99
    for _, k in ipairs(keys) do local d = D(k); if d and d < m then m = d
end end
    return m
```

```
end
function fmt(keys)
    local d = minD(keys)
    return d < 99 and string.format('%.2f', d) or '-'
end

-- ===== known obstacles: other containers + people, in the robot frame
=====
function buildObstacles()
    OB = {}
    for _, c in ipairs(containers) do
        if (c ~= cur or (state == 'travel' and not attached)) and not
(c.inRoom and not insideRoom) then
            local q = sim.getObjectPosition(c.h, body)
            table.insert(OB, {name=c.name, pts={{q[1], q[2]}}})
        end
    end
    for _, p in ipairs(people) do
        local pts = {}
        for _, h in ipairs(p.hs) do
            local q = sim.getObjectPosition(h, body)
            table.insert(pts, {q[1], q[2]})
        end
        table.insert(OB, {name=p.name, pts=pts})
    end
end
function obstacleInPath(ahead)
    local best, bestX = nil, 1e9
    for _, g in ipairs(OB) do
        for _, q in ipairs(g.pts) do
            if q[1] > 0 and q[1] < ahead and math.abs(q[2]) < CORRIDOR
and q[1] < bestX then
                best, bestX = g, q[1]
            end
        end
    end
    return best
end
function obstacleBehind(dist)
    for _, g in ipairs(OB) do
        for _, q in ipairs(g.pts) do
            if q[1] < -REAR_EDGE + 0.2 and q[1] > -(REAR_EDGE + dist) and
math.abs(q[2]) < CORRIDOR then
                return true
            end
        end
    end
    return false
end
function obstaclesInSweep()
    local front, back = false, false
    for _, g in ipairs(OB) do
        for _, q in ipairs(g.pts) do
            if q[1]*q[1] + q[2]*q[2] < TURN_SWEEP*TURN_SWEEP then
                if q[1] >= 0 then front = true else back = true end
            end
        end
    end
```

```
    return front, back
end

-- ===== actuators =====
function setState(s)
    state = s
    tState = sim.getSimulationTime()
    stuckT, stuckP, movedFwd = tState, robotPos(), false
    print('STATE -> ' .. s .. (cur and (' [' .. cur.name .. ']') or ''))
end
function setDrive(v, w)
    if v > 0.03 then movedFwd = true end
    local vl = (v - w*T/2)/R
    local vr = (v + w*T/2)/R
    for _, j in ipairs(leftWheels)  do sim.setJointTargetVelocity(j, vl)
end
    for _, j in ipairs(rightWheels) do sim.setJointTargetVelocity(j, vr)
end
end
function rearBlocked() return minD(REAR_KEYS) < REAR_STOP or
obstacleBehind(0.5) end
function setArm(liftPos, fingerPos)
    sim.setJointTargetPosition(lift, liftPos)
    sim.setJointTargetPosition(fR, fingerPos)
    sim.setJointTargetPosition(fL, fingerPos)
end
function holdArm()
    if attached then setArm(LIFT_POS, CLOSE_POS) else setArm(LIFT_POS, 0)
end
end

-- move straight AWAY from a target (reverse if it is in front, forward
if it is behind)
function moveAwayFrom(targetAhead)
    if targetAhead >= 0 then
        if rearBlocked() then setDrive(0, 0); return false end
        setDrive(-BACK_SPEED, 0)
    else
        if frontClose(1.0) then setDrive(0, 0); return false end
        setDrive(BACK_SPEED, 0)
    end
    return true
end

-- turn on the spot; if something is within the corner swing, first move
a full distance
function turnTo(w)
    local t = sim.getSimulationTime()
    local kf, kb = obstaclesInSweep()
    local tight = kf or kb or minD(FRONT_KEYS) < TURN_CLEAR or
minD(REAR_KEYS) < TURN_CLEAR
    if tight and t > graceUntil then
        local frontSide = kf or minD(FRONT_KEYS) < TURN_CLEAR
        local backSide  = kb or minD(REAR_KEYS) < TURN_CLEAR
        mr = {ret = state, start = robotPos(), dir = (backSide and not
frontSide) and 1 or -1}
        setDrive(0, 0)
        setState('makeroom')
```

```
        return
    end
    if tight then w = SLOW_TURN * sgn(w) end
    setDrive(0, w)
end

-- ===== obstacle logic =====
function blocked(targetDist, margin)
    for _, c in ipairs(FRONT_CHECKS) do
        local d = D(c[1])
        if d and d < c[2] and targetDist > d + margin then return true
end
    end
    return false
end
function frontClose(scale)
    for _, c in ipairs(FRONT_CHECKS) do
        local d = D(c[1])
        if d and d < c[2] * scale then return true end
    end
    return false
end
function startAvoid(ret, why)
    local t = sim.getSimulationTime()
    if ret == 'goto' and cur and cur.inRoom and not attached then ret =
'doorgo' end   -- room one: line up at the door again
    avoidReturn = ret
    if t > dirLock then
        avoidDir = (minD(LEFT_KEYS) >= minD(RIGHT_KEYS)) and 1 or -1
    end
    dirLock = t + 20
    av = {phase = 'back', start = robotPos(), need = AVOID_BACKUP, t0 =
t, retries = 0}
    print(string.format('obstacle (%s): reversing %.1f m, then turning
%s',
        why or '?', AVOID_BACKUP, avoidDir > 0 and 'left' or 'right'))
    setState('avoid')
end
function startBypass(g, ret)
    if ret == 'goto' and cur and cur.inRoom and not attached then ret =
'doorgo' end
    local minx, maxx, miny, maxy = 1e9, -1e9, 1e9, -1e9
    for _, q in ipairs(g.pts) do
        if q[1] > -1.0 then
            minx = math.min(minx, q[1]); maxx = math.max(maxx, q[1])
            miny = math.min(miny, q[2]); maxy = math.max(maxy, q[2])
        end
    end
    local leftLat, rightLat = maxy + BYPASS_GAP, miny - BYPASS_GAP
    local lat
    if math.abs(math.abs(leftLat) - math.abs(rightLat)) < 0.2 then
        lat = (minD(LEFT_KEYS) >= minD(RIGHT_KEYS)) and leftLat or
rightLat
    else
        lat = (math.abs(leftLat) < math.abs(rightLat)) and leftLat or
rightLat
    end
    bp = {ret = ret, i = 1,
```

```
          pts = {toWorld(math.max(minx - 0.4, 0.3), lat), toWorld(maxx +
1.2, lat)}}
    print('bypassing ' .. g.name)
    setState('bypass')
end
-- ===== walking people =====
-- a person counts as 'walking' if they moved in the last 4 s (Bill[0]
stops 3 s at each end of his path).
-- people who never move (lying / standing) are only obstacles, the robot
never waits for them.
function updatePeople(t)
    for _, p in ipairs(people) do
        local q = sim.getObjectPosition(p.hs[1], WORLD)
        if p.pos and dist2D(q, p.pos) > 0.01 then
            p.movedT = t
            p.area = p.area or {q[1], q[1], q[2], q[2]}       -- the area
this person walks in (grows as we watch)
            p.area = {math.min(p.area[1], q[1]), math.max(p.area[2],
q[1]), math.min(p.area[3], q[2]), math.max(p.area[4], q[2])}
        end
        p.pos = q
        if not p.oldT or t - p.oldT >= 1.0 then p.old, p.oldT = q, t end
    end
end
-- areas where people walk (grown by 0.8 m), learned while the simulation
runs
function walkerAreas()
    local out = {}
    for _, p in ipairs(people or {}) do
        if p.area then
            local a = p.area
            table.insert(out, {a[1] - 0.8, a[2] + 0.8, a[3] - 0.8, a[4] +
0.8})
        end
    end
    return out
end
function insideWalkerArea(q)
    for _, b in ipairs(walkerAreas()) do if boxDist(q, b) < 0.5 then
return true end end
    return false
end
function isWalking(p) return p.movedT and sim.getSimulationTime() -
p.movedT < 4 end
-- a walker near the robot; side = 'behind' (only if he is behind it) or
'any'
function walkerNear(r, side)
    for _, p in ipairs(people) do
        if isWalking(p) then
            local x, y = toRobot(p.pos[1], p.pos[2])
            if x*x + y*y < r*r and (side ~= 'behind' or x < 0.3) then
return p.name end
        end
    end
    return nil
end
-- is it unsafe to go in and grab at point q? (walker close to it, or
walking towards it)
```

```
function walkerThreat(q)
    for _, p in ipairs(people) do
        if isWalking(p) then
            local d = dist2D(p.pos, q)
            if d < WALKER_SAFE then return p.name end
            if d < WALKER_WATCH and p.old and dist2D(p.old, q) > d + 0.1
then return p.name end
        end
    end
    return nil
end
-- stand still while 'who' is set; returns true while waiting
function holdFor(who, why)
    local t = sim.getSimulationTime()
    if not who then waitSince = nil; return false end
    if not waitSince then waitSince = t; print('waiting for ' .. who .. '
' .. why) end
    if t - waitSince > WALKER_MAXWAIT then return false end
    setDrive(0, 0)
    stuckT, stuckP, movedFwd = t, robotPos(), false       -- standing
still on purpose is not 'stuck'
    tState = tState + sim.getSimulationTimeStep()          -- pause this
state's timers while waiting
    if tTarget then tTarget = tTarget + sim.getSimulationTimeStep() end
    return true
end
function checkStuck(t)
    if t - stuckT > STUCK_TIME then
        local p = robotPos()
        local moved = dist2D(p, stuckP)
        local wasFwd = movedFwd
        stuckT, stuckP, movedFwd = t, p, false
        if wasFwd and moved < 0.05 then return true end
    end
    return false
end

function checkCasualty()
    for _, pair in ipairs(CASUALTY_PAIRS) do
        local high = minD(pair[2])
        for _, k in ipairs(pair[1]) do
            local o = S[k]
            if o and o.d < 2.5 and not isContainer(o.obj) and high > o.d
+ 0.5 then
                local ok, w = pcall(sim.multiplyVector,
sim.getObjectMatrix(o.s, WORLD), o.pt)
                if ok and w then
                    local new = true
                    for _, c in ipairs(casualties) do
                        if (c.x-w[1])^2 + (c.y-w[2])^2 < 1.5^2 then new =
false; break end
                    end
                    if new then
                        local who = aliasOf(rootOf(o.obj))
                        table.insert(casualties, {x=w[1], y=w[2],
who=who})
                        print(string.format('POSSIBLE CASUALTY at x=%.2f
y=%.2f (%s)', w[1], w[2], who))
```

```
                    end
                end
            end
        end
    end
end

function driveTo(tx, ty, vmax)
    local p = robotPos()
    local dist = dist2D(p, {tx, ty})
    local e = wrapAngle(math.atan2(ty - p[2], tx - p[1]) - robotYaw())
    if frontClose(SLOWDOWN) then vmax = math.min(vmax, SLOW_SPEED) end
    if math.abs(e) > 0.4 then turnTo(limit(1.5*e, -TURN_SPEED,
TURN_SPEED))
    else setDrive(limit(0.8*dist, 0.1, vmax), limit(2*e, -0.8, 0.8)) end
    return dist
end

-- stand still until LEAVE_AT (or the same moment of his next walk round,
if it is already past it)
function syncWait(t)
    if LEAVE_AT and not syncTarget then
        syncTarget = LEAVE_AT
        while syncTarget < t - 0.5 do syncTarget = syncTarget + MAN_CYCLE
end
        print(string.format('waiting %.1f s, so it leaves for %s at
t=%.2f s like the 0.25 code', syncTarget - t, MAN_CONTAINER, syncTarget))
    end
    if LEAVE_AT and t < syncTarget then
        stuckT, stuckP = t, robotPos()               -- standing still on
purpose is not 'stuck'
        return true
    end
    if not leavePrinted then
        leavePrinted = true
        print(string.format('LEAVING for %s at t=%.2f s', MAN_CONTAINER,
t))
    end
    return false
end
function pickNextContainer()
    local p = robotPos()
    local best, bestD = nil, 1e9
    for _, c in ipairs(containers) do
        if not c.done and not c.extra then
            local d = dist2D(p, sim.getObjectPosition(c.h, WORLD))
            if d < bestD then best, bestD = c, d end
        end
    end
    if best then return best end
    -- NEW: the other 8 are done -> the building one, then the room one
    for _, name in ipairs({BUILDING_CONTAINER, ROOM_CONTAINER}) do
        for _, c in ipairs(containers) do
            if c.name == name and not c.done then return c end
        end
    end
    return nil
end
```

```
function chooseSlot()
    local p = robotPos()
    local best, bestD = nil, -1e9
    for _, s in ipairs(slots) do
        if not s.used then
            local free = true
            for _, c in ipairs(containers) do
                if c ~= cur then
                    local q = sim.getObjectPosition(c.h, WORLD)
                    if (q[1]-s.x)^2 + (q[2]-s.y)^2 < 0.6^2 then free =
false; break end
                end
            end
            if free then
                -- fill from the far (wall) side first: the robot always
comes in from the open side
                local d = -((s.x - zp[1])*ENTRY[1] + (s.y -
zp[2])*ENTRY[2])
                if d > bestD + 0.01 then best, bestD = s, d end
            else
                s.used = true
            end
        end
    end
    return best
end

function printSummary()
    print('===== BIOHAZARD LOG =====')
    print('name, picked_x, picked_y, placed_x, placed_y, upright,
in_green_area')
    for _, r in ipairs(records) do
        print(string.format('%s, %.2f, %.2f, %.2f, %.2f, %s, %s',
            r.name, r.px, r.py, r.dx, r.dy,
            r.upright and 'yes' or 'NO', r.inZone and 'yes' or 'NO'))
    end
    print(string.format('delivered %d container(s)', #records))
    print('===== POSSIBLE CASUALTIES =====')
    for _, c in ipairs(casualties) do
        print(string.format('x=%.2f y=%.2f (%s)', c.x, c.y, c.who))
    end
end

-- ===== main loop =====
function step()
    local t = sim.getSimulationTime()
    local dt = t - tState
    S = readAll()
    updatePeople(t)
    -- diagonal rays: DIAG_DIST, and 0.1 for the room container (through
the narrow doorway)
    local dg = (cur and cur.name == ROOM_CONTAINER) and DOOR_DIAG_DIST or
DIAG_DIST
    FRONT_CHECKS[7][2], FRONT_CHECKS[8][2] = dg, dg
    buildObstacles()

    if t - lastC > 0.2 then lastC = t; checkCasualty() end
```

```
    if t - lastP > 1 then
        lastP = t
        local p = robotPos()
        print(string.format('state=%s x=%.2f y=%.2f | front=%s
lowFront=%s left=%s right=%s rear=%s',
            state, p[1], p[2],
            fmt({'F', 'FL', 'FR', 'DFL', 'DFR'}), fmt({'LF', 'LL',
'LR'}),
            fmt({'SL', 'LSL'}), fmt({'SR', 'LSR'}), fmt(REAR_KEYS)))
    end

    -- climb mode: on a step, drive straight with nothing interrupting
    local pitch = sim.getObjectMatrix(body, WORLD)[9]
    if CLIMB_STATES[state] and math.abs(pitch) > CLIMB_PITCH then
        if not climbing then
            climbing, climbT0 = true, t
            print(string.format('climb mode (%s)', pitch > 0 and 'going
up' or 'going down'))
        end
        if t - climbT0 < CLIMB_MAX and minD({'F'}) > 0.3 then
            holdArm()
            setDrive(pitch > 0 and CLIMB_UP or CLIMB_DOWN, 0)
            stuckT, stuckP, movedFwd = t, robotPos(), false
            return
        end
    else
        climbing = false
    end

    if state == 'start' then
        setDrive(0, 0); setArm(0, 0)
        if dt > 1 then setState('pick') end

    elseif state == 'pick' then
        setDrive(0, 0); setArm(LIFT_POS, 0)
        cur = pickNextContainer()
        if not cur then
            setState('finished')
        elseif cur.name == MAN_CONTAINER and syncWait(t) then
            -- standing still so it sets off at the same moment as the
0.25 code
        elseif cur.inRoom and door and not insideRoom then
            tTarget = t; doorDir, doorRet = 'in', 'goto'      -- NEW:
room container: go in through the door
            startTravel(door.out, door.block, 'doorface')
        elseif cur.approach then
            tTarget = t; startTravel(cur.approach, cur.block, 'goto')   -
- NEW: building container: open side first
        else
            tTarget = t
            local q = sim.getObjectPosition(cur.h, WORLD)
            local w = (not insideWalkerArea(q)) and
nextWaypoint(robotPos(), {q[1], q[2]}) or nil
            if w and dist2D(w, q) > 0.5 then
                -- a walking person's area is in the way: go round it,
then come in straight
                local dx, dy = w[1] - q[1], w[2] - q[2]
```

```
                local L = math.sqrt(dx*dx + dy*dy)
                local a = math.min(2.5, L)
                print('going round the walking area to ' .. cur.name)
                startTravel({q[1] + dx/L*a, q[2] + dy/L*a}, nil, 'goto')
            else
                setState('goto')
            end
        end

    elseif state == 'goto' then
        local c = sim.getObjectPosition(cur.h, body)
        local cx, cy = c[1], c[2]
        local d = math.sqrt(cx*cx + cy*cy)
        local e = math.atan2(cy, cx)
        if d > 1.5 then setArm(LIFT_POS, 0) else setArm(0, 0) end
        local inPath = (math.abs(e) < 0.4) and obstacleInPath(cx - 0.3)
or nil

        if d < 3.5 and d > 1.3 and
holdFor(walkerThreat(sim.getObjectPosition(cur.h, WORLD)), 'to walk away
from ' .. cur.name) then
            -- the walking man is near the container or coming towards
it: wait, then go in and out quickly
        elseif inPath and cur.name == MAN_CONTAINER and inPath.name ==
MAN_NAME
               and holdFor(MAN_NAME, 'to walk out of the way, then
straight to ' .. cur.name) then
            -- Biohazard10: the man is in the way: wait for him, then go
straight to it (no driving round / past it)
        elseif inPath and not (cur.name == MAN_CONTAINER and inPath.name
== MAN_NAME) then
            startBypass(inPath, 'goto')
        elseif math.abs(e) < 0.4 and blocked(d, 0.8) then
            startAvoid('goto', 'sensor')
        elseif checkStuck(t) then
            startAvoid('goto', 'stuck')
        elseif cx < HOLD_X + 0.02 and cx > HOLD_X - 0.12 and math.abs(cy)
< 0.04 then
            setDrive(0, 0); setState('grasp')
        elseif d < 1.0 and (math.abs(cy) > 0.10 or cx < HOLD_X - 0.12)
then
            setState('reposition')
        elseif math.abs(e) > 0.3 then
            turnTo(limit(1.5*e, -TURN_SPEED, TURN_SPEED))
        else
            local vmax = frontClose(SLOWDOWN) and SLOW_SPEED or
DRIVE_SPEED
            -- fast when far, slows down automatically as it gets close
            setDrive(limit(0.8*(cx - HOLD_X), 0.05, vmax), limit(2.5*e, -
0.8, 0.8))
        end
        if t - tTarget > 300 then
            print('giving up on ' .. cur.name)
            cur.done = true; setState('pick')
        end

    elseif state == 'reposition' then
        setArm(LIFT_POS, 0)
```

```
        local c = sim.getObjectPosition(cur.h, body)
        local d = math.sqrt(c[1]^2 + c[2]^2)
        if d >= REPOS_DIST or dt > backTime(REPOS_DIST) then
            setState('goto')
        elseif not moveAwayFrom(c[1]) then
            graceUntil = t + 5; setState('goto')
        end

    elseif state == 'makeroom' then
        holdArm()
        local moved = dist2D(robotPos(), mr.start)
        local blockedDir = (mr.dir < 0 and rearBlocked()) or (mr.dir > 0
and frontClose(0.6))
        if moved >= MAKEROOM or blockedDir or dt > backTime(MAKEROOM)
then
            graceUntil = t + 5
            setState(mr.ret)
        else
            setDrive(BACK_SPEED * mr.dir, 0)
        end

    elseif state == 'bypass' then
        holdArm()
        local wp = bp.pts[bp.i]
        if frontClose(1.0) then
            startAvoid(bp.ret, 'blocked while bypassing')
        elseif driveTo(wp[1], wp[2], attached and CARRY_SPEED or
DRIVE_SPEED) < 0.3 then
            bp.i = bp.i + 1
            if bp.i > #bp.pts then setState(bp.ret) end
        end
        if dt > 45 then setState(bp.ret) end

    elseif state == 'avoid' then
        holdArm()
        local p = robotPos()
        if av.phase == 'back' then
            local backed = dist2D(p, av.start)
            if backed >= av.need or rearBlocked() or t - av.t0 >
backTime(av.need) then
                print(string.format('reversed %.1f m, now turning',
backed))
                av.phase, av.yaw0, av.t0 = 'turn', robotYaw(), t
            else
                setDrive(-BACK_SPEED, 0)
            end
        elseif av.phase == 'turn' then
            local turned = math.abs(wrapAngle(robotYaw() - av.yaw0))
            local kf, kb = obstaclesInSweep()
            if turned >= AVOID_TURN or t - av.t0 > 12 then
                av.phase, av.start, av.t0 = 'forward', p, t
            elseif (kf or kb) and av.retries < 2 and not rearBlocked()
then
                av.retries = av.retries + 1
                av.phase, av.start, av.need, av.t0 = 'back', p,
AVOID_BACKUP / 2, t
            else
```

```
                setDrive(0, ((kf or kb) and SLOW_TURN or TURN_SPEED) *
avoidDir)
            end
        elseif av.phase == 'forward' then
            if frontClose(1.0) or obstacleInPath(FRONT_EDGE + 0.6) then
                startAvoid(avoidReturn, 'still blocked')
            elseif dist2D(p, av.start) >= DETOUR_DIST or t - av.t0 > 12
then
                setState(avoidReturn)
            else
                setDrive(DETOUR_SPEED, 0)
            end
        end

    elseif state == 'grasp' then
        setDrive(0, 0); setArm(0, CLOSE_POS)
        if dt > 2.5 then
            local q = sim.getObjectPosition(cur.h, WORLD)
            cur.px, cur.py = q[1], q[2]
            print(string.format('PICKED %s at x=%.2f y=%.2f', cur.name,
q[1], q[2]))
            attachContainer(cur.h)
            setState('lift')
        end

    elseif state == 'lift' then
        setDrive(0, 0); setArm(LIFT_POS, CLOSE_POS)
        if dt > 3 then
            dropSlot = chooseSlot()
            if not dropSlot then
                print('no free spot left in the green area, using the
centre')
                dropSlot = {x=zp[1], y=zp[2]}
            end
            dropSlot.used = true
            print(string.format('carrying to green area spot x=%.2f
y=%.2f', dropSlot.x, dropSlot.y))
            pickPos = robotPos()
            setState('retreat')
        end

    elseif state == 'retreat' then
        -- after lifting: reverse straight back out the way we came in,
THEN turn
        setArm(LIFT_POS, CLOSE_POS)
        local backed = dist2D(robotPos(), pickPos)
        local backNeed = (cur.name == MAN_CONTAINER) and MAN_BACKUP or
PICK_BACKUP   -- 10 m for Biohazard10
        if holdFor(walkerNear(WALKER_NEAR, 'behind'), 'to pass behind')
then
            -- walker behind: don't reverse into him
        elseif backed >= backNeed or dt > backTime(backNeed) then
            print(string.format('backed out %.1f m, now heading for the
green area', backed))
            afterRetreat()
        elseif minD({'B', 'LB', 'BL', 'BR'}) < EXIT_STOP or
obstacleBehind(0.3) then
            setDrive(0, 0)
```

```
            if dt > 3 then afterRetreat() end
        else
            setDrive(-BACK_SPEED, 0)
        end

    elseif state == 'carry' then
        setArm(LIFT_POS, CLOSE_POS)
        local sx, sy = toRobot(dropSlot.x, dropSlot.y)
        local dist = math.sqrt(sx*sx + sy*sy)
        local e = math.atan2(sy, sx)
        local inPath = (math.abs(e) < 0.4 and dist > HOLD_X + 0.3) and
obstacleInPath(sx - 0.3) or nil

        if inPath then
            startBypass(inPath, 'carry')
        elseif math.abs(e) < 0.4 and dist - HOLD_X > 0.3 and blocked(dist
- HOLD_X, 0.5) then
            startAvoid('carry', 'sensor')
        elseif checkStuck(t) then
            startAvoid('carry', 'stuck')
        elseif sx > 0 and sx <= HOLD_X + 0.03 and math.abs(sy) < 0.25
then
            setDrive(0, 0); setState('lower')
        elseif dist < FINAL_APPROACH then
            if sx > HOLD_X + 0.03 and math.abs(e) < 0.6 then
                setDrive(limit(0.5*(sx - HOLD_X), 0.05, 0.15),
limit(1.5*e, -0.3, 0.3))
            else
                setState('carryback')
            end
        elseif math.abs(e) > 0.4 then
            if not holdFor(walkerNear(WALKER_NEAR, 'any'), 'to pass
before turning') then turnTo(limit(1.5*e, -TURN_SPEED, TURN_SPEED)) end
        else
            local vmax = frontClose(SLOWDOWN) and SLOW_SPEED or
CARRY_SPEED
            setDrive(limit(0.8*(dist - HOLD_X), 0.1, vmax), limit(2*e, -
0.8, 0.8))
        end

    elseif state == 'carryback' then
        setArm(LIFT_POS, CLOSE_POS)
        local sx, sy = toRobot(dropSlot.x, dropSlot.y)
        local dist = math.sqrt(sx*sx + sy*sy)
        if dist >= CARRYBACK_DIST or dt > backTime(CARRYBACK_DIST) then
            graceUntil = t + 5; setState('carry')
        elseif not moveAwayFrom(sx) then
            graceUntil = t + 5; setState('carry')
        end

    elseif state == 'lower' then
        setDrive(0, 0); setArm(0, CLOSE_POS)
        if dt > 3 then detachContainer(cur.h); setState('release') end

    elseif state == 'release' then
        setDrive(0, 0); setArm(0, 0)
        if dt > 2.5 then
            local q = sim.getObjectPosition(cur.h, WORLD)
```

```
            local m = sim.getObjectMatrix(cur.h, WORLD)
            local upright = m[11] > 0.9
            local inside = inZone(q[1], q[2])
            print(string.format('PLACED %s at x=%.2f y=%.2f upright=%s
in_green_area=%s',
                cur.name, q[1], q[2], upright and 'yes' or 'NO', inside
and 'yes' or 'NO'))
            table.insert(records, {name=cur.name, px=cur.px, py=cur.py,
                                   dx=q[1], dy=q[2], upright=upright,
inZone=inside})
            cur.done = true
            cur = nil
            dropPos = robotPos()
            setState('exit')
        end

    elseif state == 'exit' then
        local p = robotPos()
        local backed = dist2D(p, dropPos)
        if backed < 0.6 then setArm(0, 0) else setArm(LIFT_POS, 0) end
        if holdFor(walkerNear(WALKER_NEAR, 'behind'), 'to pass behind')
then
            -- walker behind: don't reverse into him
        elseif backed >= MIN_BACKUP and not inZone(p[1], p[2],
EXIT_CLEAR) then
            setDrive(0, 0)
            print(string.format('backed up %.1f m straight', backed))
            setState('pick')
        elseif minD({'B', 'LB'}) < EXIT_STOP then
            setDrive(0, 0)
            if dt > 20 then print('exit blocked behind, carrying on');
setState('pick') end
        else
            setDrive((backed < 0.6) and -0.1 or -BACK_SPEED, 0)
        end

    -- ===== NEW states for the two extra containers =====
    elseif state == 'travel' then
        holdArm()
        local p = robotPos()
        if tr.entered ~= tState then                          -- (re)plan
each time we come back here
            tr.entered = tState; tr.wp = nextWaypoint(p, tr.goal)
        end
        local wp = tr.wp
        local d = dist2D(p, wp)
        local lx, ly = toRobot(wp[1], wp[2])
        local e = math.atan2(ly, lx)
        local inPath = (math.abs(e) < 0.4) and
obstacleInPath(math.min(lx, d) - 0.2) or nil
        if inPath then
            startBypass(inPath, 'travel')
        elseif math.abs(e) < 0.4 and blocked(d, 0.8) then
            startAvoid('travel', 'sensor')
        elseif checkStuck(t) then
            startAvoid('travel', 'stuck')
        elseif math.abs(e) > 0.4 and holdFor(walkerNear(WALKER_NEAR,
'any'), 'to pass before turning') then
```

```
            -- walker next to us: wait before turning
        elseif driveTo(wp[1], wp[2], attached and CARRY_SPEED or
DRIVE_SPEED) < 0.4 then
            if wp == tr.goal then setDrive(0, 0); tTarget = t;
setState(tr.ret)
            else tr.wp = nextWaypoint(p, tr.goal) end      -- round the
corner: plan the next bit
        end
        if state == 'travel' and t - tTarget > 300 and not attached then
            print('giving up on ' .. cur.name); cur.done = true;
setState('pick')
        end

    elseif state == 'doorgo' then
        doorDir = 'in'
        startTravel(door.out, door.block, 'doorface')

    elseif state == 'carryhome' then
        carryHome()

    elseif state == 'doorface' then
        holdArm()
        local s = (doorDir == 'in') and -1 or 1
        local e = wrapAngle(math.atan2(s*door.ny, s*door.nx) -
robotYaw())
        if math.abs(e) < 0.08 then setDrive(0, 0); setState('doorwait')
        else setDrive(0, limit(1.5*e, -SLOW_TURN, SLOW_TURN)) end

    elseif state == 'doorwait' then
        holdArm()
        if doorOpen() then
            print('door open, going in for ' .. cur.name)
            tTarget = t
            setState('goto')                                    --
Biohazard4 is right behind the door
        elseif dt > 30 then
            print('door did not open')
            cur.done = true; setState('pick')
        else
            -- creep up to the door, centred on it, so its sensors see
us; stop 0.3 m before it
            local p = robotPos()
            local s = (doorDir == 'in') and -1 or 1
            local hx, hy = s*door.nx, s*door.ny                 --
direction through the door
            local gap = -((p[1] - door.c[1])*hx + (p[2] - door.c[2])*hy)
            local lat = hx*(p[2] - door.c[2]) - hy*(p[1] - door.c[1])
            local e = wrapAngle(math.atan2(hy, hx) - robotYaw())
            if gap > FRONT_EDGE + 0.3 and minD({'F', 'LF'}) > 0.25 then
                setDrive(0.12, limit(2*e - 1.5*lat, -0.4, 0.4))
            else
                setDrive(0, 0)
            end
        end

    elseif state == 'doorpass' then
        holdArm()
        local goal = (doorDir == 'in') and door.inn or door.out
```

```
        if driveTo(goal[1], goal[2], 0.3) < 0.4 or dt > 30 then
            setDrive(0, 0)
            insideRoom = (doorDir == 'in')
            print('through the door: robot is ' .. (insideRoom and
'INSIDE' or 'OUTSIDE') .. ' the room')
            tTarget = t
            setState(doorRet)
        end

    elseif state == 'finished' then
        setDrive(0, 0); setArm(0, 0)
        if not summaryDone then printSummary(); summaryDone = true end
    end
end

function sysCall_actuation()
    if not ready then return end
    local ok, err = xpcall(step, errText)
    if not ok then
        ready = false
        setDrive(0, 0)
        print('RUN ERROR: ' .. tostring(err))
    end
end
```